

Inigo Quilez :: articles :: premultiplied alpha - 2022



I'm aware that enough has been said about Premultiplied Alpha vs Not-Premultiplied Alpha blending and compositing, so this article won't try to teach you about premultiplied alpha or preach its benefits or correctness. Although if you do need to learn about that or get a refreshed, then I recommend you visit [Tom Forsyth's great and classic article](#), then come back here to continue reading.

In this article I just wanted to give a mathematical characterization to the problem of "black edges" when you render an image or texture that has black transparent background and some opaque pixels. As we might know from experience, things go wrong when using regular alpha blending and you start scaling up or down such image, although things also go wrong when any linear pixel operation happens, such as mipmapping, filtering with a low pass filter such a blur, etc.

Now, we also know that one workaround is to dilate the color of the solid bits of the texture outwards to the black pixels. This might or might not be easy to do, but we also know it's unnecessary under premultiplied alpha, which is easy enough to do. In this article we'll explore why and how exactly both of these approaches work.

So for now let's assume you are doing the right thing here, that is, we are using premultiplied alpha blending and hence our blending equation for a texture color c and alpha value a and a background destination color d is

$$B_p(c,a,d) = c + (1-a) \cdot d$$

as opposed to regular alpha blending

$$B_r(c,a,d) = c \cdot a + (1-a) \cdot d$$

Note I used the p and r subscript to differentiate "premultiplied" and "regular" alpha-blending. Now let's also assume that we are doing some simple bilinear scaling of our texture, and keep the math simple let's assume this is 1D scaling only. In other words, any two consecutive texels of our texture will generate a gradient of colors and alphas during rendering, of the form

$$S(c) = c_1 + x \cdot (c_2 - c_1)$$

$$S(a) = a_1 + x \cdot (a_2 - a_1)$$

That is, the color **c** will transition from c_1 to c_2 as x goes from 0 to 1 across the pixels of the destination buffer **d**, and alpha **a** will also transition from a_1 to a_2 , linearly. So far, pretty simple. And let's do things even simpler if we assume the background **d** is of constant color, although everything we'll prove still works if we don't assume this.

So, now this is the key observation for this article: we expect scaling and then blending the texture into the screen (which is the normal order of operations) to result in the same picture as if we were blending the texture with the background first and then scaling the result. In other words, we want

$$B_p(S(cp), S(a), d) = S(B_p(cp, a, d))$$

Or yet in other words, we want that blending a scaled image is the same as scaling a blended image.

The math of Premultiplied Alpha Blending

So, let's expand the formulas and see if we can prove this to be true:

$$\begin{aligned} B_p(S(c), S(a), d) &= \\ S(c) + (1 - S(a)) \cdot d &= \\ c_1 + x \cdot (c_2 - c_1) + d \cdot (1 - a_1 - x \cdot (a_2 - a_1)) &= \\ c_1 + d \cdot (1 - a_1) + x \cdot [c_2 - c_1 - d \cdot (a_2 - a_1)] &= \\ B_p(c_1, a_1, d) + x \cdot [c_2 - d \cdot a_2 + d - (c_1 - d \cdot a_1 + d)] &= \\ B_p(c_1, a_1, d) + x \cdot [B_p(c_2, a_2, d) - B_p(c_1, a_1, d)] &= \\ S(B_p(c, a, d)) \end{aligned}$$

Or in short

$$B_p(S(c), S(a), d) = S(B_p(c, a, d))$$

So indeed, blending interpolated pixels is the same as interpolating blended pixels. Fantastic, pre-multiplied alpha blending works!



The math of Regular Alpha Blending

Now, let's see why and how regular alpha blending does not work. Recall our blending equations is now $B_r(c, a, d) = c \cdot a + (1-a) \cdot d$, so

$$\begin{aligned} B_r(S(c), S(a), d) &= \\ S(c)S(a) + d \cdot (1-S(a)) &= \\ [c_1 + x \cdot (c_2 - c_1)] \cdot [a_1 + x \cdot (a_2 - a_1)] + d \cdot [1 - a_1 - x \cdot (a_2 - a_1)] &= \\ c_1 a_1 + d \cdot (1 - a_1) + c_1 x \cdot (a_2 - a_1) + a_1 x \cdot (c_2 - c_1) + x^2 \cdot (c_2 - c_1)(a_2 - a_1) - & \\ x \cdot d \cdot (a_2 - a_1) &= \\ B_r(c_1, a_1, d) + x \cdot [c_1 a_2 + a_1 c_2 - a_1 c_1 - a_2 c_2 + B(c_2, a_2, d) - & \\ B(c_1, a_1, d)] + x^2 \cdot (c_2 - c_1)(a_2 - a_1) &= \\ B_r(c_1, a_1, d) + x \cdot [B_r(c_2, a_2, d) - B_r(c_1, a_1, d)] + x \cdot [c_1 a_2 + a_1 c_2 - a_1 c_1 - & \end{aligned}$$

$$a_2c_2] + x^2 \cdot (c_2 - c_1)(a_2 - a_1) =$$

$$S(B_r(c, a, d)) + x \cdot (c_1a_2 + a_1c_2 - a_1c_1 - a_2c_2) + x^2 \cdot (c_2 - c_1)(a_2 - a_1) =$$

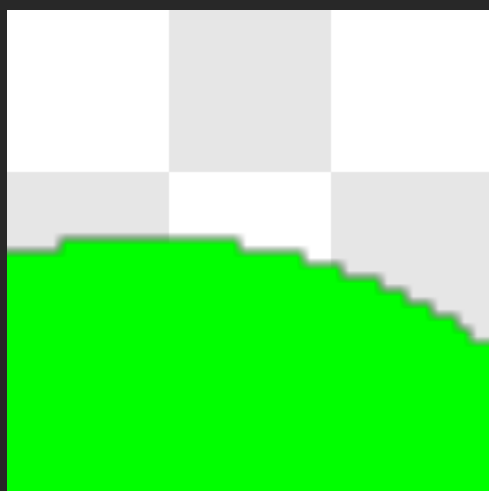
$$S(B_r(c, a, d)) + x \cdot [c_1a_2 + a_1c_2 - a_1c_1 - a_2c_2 + x \cdot (c_2a_2 - c_2a_1 - c_1a_2 + c_1a_1)] =$$

$$S(B_r(c, a, d)) + x(1-x) \cdot (c_1a_2 + a_1c_2 - a_1c_1 - a_2c_2)$$

Summarizing, we have that

$$B_r(S(c), S(a), d) = S(B_r(c, a, d)) + x(1-x) \cdot (c_1a_2 + a_1c_2 - a_1c_1 - a_2c_2)$$

That is, the blending of interpolated pixels is the same as the interpolation of the blended pixels PLUS some weird stuff that depends on the pixels we are interpolating and the interpolation factor x . And that's naturally a problem. But let's analyze that “weird stuff” a bit to see how exactly things get wrong here and it produces a black border rather than some other thing.



The error term

For example, the shape of the problem is a parabola: $x \cdot (1-x)$, and this parabola will cancel when $x=0$ or $x=1$, which is exactly at the exact pixel positions we are interpolating between. So, the colors of the pixels will be alright at exact pixel locations, it's between pixel locations that things will look wrong, and this will peak at the center of the parabola which happens at $x=0.5$, right between the two pixels.

We know this wrongness often manifest as the infamous dark edge, so let's see why that is the shape and color of our error:

Recall the error is

$$e = x \cdot (1-x) \cdot (c_1 \cdot a_2 + a_1 \cdot c_2 - a_1 \cdot c_1 - a_2 \cdot c_2)$$

So first we note that the error will be zero for all x , meaning that everything will be correct even between pixels, if $a_1=a_2$

Since in that case $c_1 \cdot a_2$ cancels out with $-c_1 \cdot a_1$ and $c_2 \cdot a_1$ cancels out with $-c_2 \cdot a_2$.

So, we won't see rendering artifacts in the solid parts of the texture, nor in the fully transparent areas of the texture. It's in the transitions between opaque and transparent that the error term e will be non-zero. This checks with experience:

Similarly, if the alpha channel does change but the color channel doesn't, then $c_1=c_2$ and in that case $c_1 \cdot a_1$ cancels out with $-c_2 \cdot a_1$ and $c_1 \cdot a_2$ cancels out with $-c_2 \cdot a_2$, and so $e=0$ and then too we won't see an error. This corresponds to the hack of dilating the pixel colors of the billboards outwards into the transparent areas. Again, the math checks with what we know from experience.

The failure is when either the alpha channel changes (ie, $a_1 \neq a_2$) and the color channel changes (ie, $c_1 \neq c_2$), because then nothing cancels out, e is always bigger than zero except for the exact pixel center $x=0$ and $x=1$, and so we see the error.

Furthermore, assuming we are transitioning for example from a green leaf ($c_1=\text{green}$, $a_1=1$) to a black transparent background ($c_2=\text{black}$, $a_2=0$), then the error term will be:

$$e = -x \cdot (1-x) \cdot c_1$$

while if we are transitioning the other way from a black transparent background (c_1 =black, a_1 =zero) to the leaf (c_2 =green, a_2 =1) we get

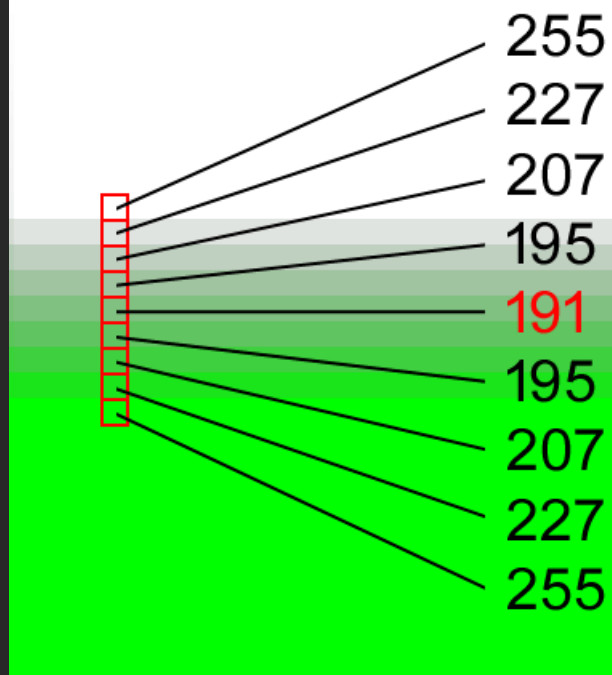
$$e = -x \cdot (1-x) \cdot c_2$$

In both cases the error is a negative color contribution to the rendering, stronger the brighter our leaf color is. In the worst case scenario, right in the middle of the transition, at $x=0.5$, this error will be:

$$e = -c_2/4$$

For example, in the pure green to white transition in the picture below, this means the error is $-255/4$ and therefore the color at the darkest point in the transition will be $255 - 255/4 = 191$ instead of the expected 255, which matches the empirical measurement.

I probably don't need to justify myself if I claim that non-premultiplied alpha blending will go horribly wrong in the case of general 2D bilinear interpolation given that just the 1D linear interpolation misbehaves.



I'm aware there's no actionable advice in this article, other than keep doing what you probably were already doing in terms of premultiplied blending, but I hope you enjoyed the little mathematical take on it.

Lastly, here's the source code used to make the pictures in the article, which shows how to perform the correct premultiplied alpha blending: <https://www.shadertoy.com/view/cds3z7M>

inigo quilez - learning computer graphics since 1994